Interarrival Time (IAT) Calculator

AI Build Prompts — Consolidated Log

Every prompt used to build, test, and fix this project in Google AI Studio (Build mode), in order

Why This Document Exists

This tool was built by directing an AI code-generation platform (Google AI Studio) with structured, plain-English prompts rather than writing code directly. Each prompt below represents a real design decision or a real bug found during testing - not a single “build me an app” request. The point of including this log is to show the actual engineering judgment behind the tool: what was specified, what went wrong, how it was diagnosed, and how the fix was scoped.

Prompt 1 — Initial Build

The starting prompt, after the calculation logic (site-type-specific formulas, new-item substitution rules) had been fully worked out on paper and a hand-built sample dataset was ready to test against.

```
Build a web app called "Interarrival Time (IAT) Calculator" that uploads the
following 4 CSV files: site_master.csv, sku_master.csv,
movement_type_reference.csv, and movement_data.csv.

Let me set a "Snapshot Date" input (default 2026-08-10).

Then calculate Interarrival Time per SKU-Site using this logic:

1. Lookback window = 182 days before the snapshot date.
2. A goods receipt movement is any row in movement_data.csv within the
   window whose MovementType is marked IsGoodsReceipt = Y for that region
   in movement_type_reference.csv.
3. A SKU-Site is "new" if it has zero goods receipt movements in the
   window, otherwise it's "old".
4. For OLD SKU-Sites: look up SiteLocationType from site_master.csv.
   - If PDC: count distinct ISO weeks (within the window) that have at
     least one goods receipt for that SKU-Site. IAT = 182 / that count.
   - If SDC: count distinct calendar days (within the window) that have
     at least one goods receipt for that SKU-Site. IAT = 182 / that count.
5. For NEW SKU-Sites, try in order and stop at the first that works:
   a. Find other SKUs with the same GTIN that already have a calculated
      IAT at the same site. If found, average their IAT and use that.
   b. Otherwise find other SKUs with the same Segment that have a
      calculated IAT at the same site, and average those.
   c. Otherwise assign a default IAT of 182 days.

Show a results table with SKU, Site, SKU Description, Site Description,
Interarrival Time (days), and a "Logic Used" column explaining which
rule produced each number.
```

Outcome: First working version produced. Testing against the hand-built answer key surfaced Defects 1-3 (see UAT log): SKU/Site columns swapped, wrong formula applied to Hub sites, and SKUs evaluated against sites outside their region.

Prompt 2 — Fix: Data Mapping, Formula Selection, Region Scoping

Written after cross-checking the first version's output against the hand-built answer key and finding SKU/Site columns swapped, the wrong formula used for Hub sites, and cross-region SKU-Site pairs being evaluated.

```
Fix the following issues in the IAT Calculator app:

1. SKU/Site mapping bug: There are currently two separate code paths for
   calculating Interarrival Time, and they disagree with each other. Delete
   the duplicate/second calculation logic entirely and use ONE single
```

```
   function for all SKU-Site pairs. Always read MaterialNumber as the SKU
   and PlantCode as the Site from movement_data.csv - never swap them.

2. PDC vs SDC formula bug: The formula must branch on SiteLocationType from
   site_master.csv for every SKU-Site pair, not default to one formula.

3. Scope the SKU-Site universe correctly: only generate a SKU-Site pair to
   evaluate if the SKU's RegionId matches the Site's RegionId.

4. Remove the "Default (2026-08-10)" and "2026-06-30" quick-select buttons
   next to the Snapshot Date field - keep only the calendar date picker.

5. Remove the "Match Regions (SKU & Site)" checkbox/filter entirely - it's
   unnecessary now that scoping is handled correctly in fix #3.

6. Make the summary ribbon (Total Pairs, Old vs New, Avg IAT, Substitution
   Rules Used) recalculate based on the CURRENTLY FILTERED rows in the
   table, not the full unfiltered dataset.
```

Outcome: All 4 mapping/formula/scoping issues resolved and verified. Ribbon and filters became reactive. A separate, subtler defect was found next: substitution logic was allowed to chain off other non-real values (Defect 4).

Prompt 3 — Fix: Substitution Chaining

Found while spot-checking new-item results: a new SKU's estimated value was being built from another SKU's already-estimated (non-real) value, which lets small errors compound the further an item is from real data.

```
Fix the substitution chaining bug: GTIN Substitution and Hierarchy
Substitution must ONLY average sibling SKUs whose IAT came from Direct
Calculation (real movement history at that site) - never a sibling whose
own IAT came from GTIN Substitution, Hierarchy Substitution, or the
Default fallback. Do not let substitutions chain off other substitutions.

Concretely: when evaluating a candidate sibling, check its own rule
source. If that source is anything other than Direct Calculation, exclude
it and continue checking other candidates (or fall through to the next
rule / default if none qualify).

Recalculate everything and verify SKU1060 becomes 20.0 days (average of
ONLY the two siblings with real data, excluding the one borrowed value).
```

Outcome: Fixed and re-verified. This was the last correctness defect in the core calculation engine - all 16 test cases have passed consistently since.

Prompt 4 — Feature: AI Insights Agent

Added after the core calculation was fully verified, to make the tool explain its own numbers rather than just display them. Deliberately designed so the AI narrates pre-computed facts and never performs the arithmetic itself.

```
Add an AI Insights chat panel to this dashboard. Let me type natural-
language questions and get grounded answers using Gemini function calling.
Do NOT let the model calculate any IAT number itself - it may only call
the following tools and narrate their results.

Define three tools:

1. explain_sku_site(sku, site) - looks up the already-calculated record for
   that SKU-Site pair and returns the formula used, the qualifying count,
   the actual receipt dates, and (if substitution was used) which rule and
   which sibling SKU(s) were involved.

2. query_results(filter_criteria, sort_by, direction, top_n) - runs an
```

```
   actual filter/sort against the current results table IN CODE, not via
   the LLM, and returns the resulting rows.

3. simulate_snapshot(hypothetical_date) - re-runs the SAME calculation
   function the main dashboard uses (do not duplicate the logic) with a
   different snapshot date, and returns a diff against the current
   results. This must be read-only and never alter the live dashboard.

System instructions: always call the relevant tool(s) before answering;
never state a number that didn't come from a tool result; if a question
needs more than one tool, call multiple and combine the results.
```

Outcome: Working AI agent added, able to answer single-record 'why' questions, aggregate/ranking questions, and what-if snapshot-date simulations, all grounded in the same calculation engine as the dashboard.

Prompt 5 — Feature: Business Impact / Cycle Stock Tab

Added to turn the raw KPI into a quantified business consequence - showing what each SKU's real delivery pattern means for how much inventory buffer is actually needed, versus a naive planning assumption.

```
Add a new tab called "Business Impact" that reads the already-calculated
IAT for each SKU-Site and combines it with a new uploaded file,
demand_and_cost.csv (SKU, Site, Description, DailyDemandUnits, UnitCostUSD,
BaselineLeadTimeDays).

For every SKU-Site present in that file, calculate:
- Cycle Stock (Actual) = DailyDemandUnits x Actual IAT / 2
- Cycle Stock (Naive) = DailyDemandUnits x BaselineLeadTimeDays / 2
- Delta Units and Delta Cost (Delta Units x UnitCostUSD)
- Risk Flag: Understated (stockout risk) / Overstated (excess carrying
  cost) / No gap, based on the sign of Delta Units

Display as a sortable table plus a summary ribbon (total $ understated,
total $ overstated, net $ impact) that updates live with filters, and a
bar chart comparing Actual vs Naive cycle stock per SKU-Site.
```

Outcome: Tab built, but testing found it was fabricating demand/cost figures instead of reading the uploaded file (Defect 5), which also inverted three risk flags (Defect 6) - addressed in the next prompt.

Prompt 6 — Fix: Data Fidelity & Professional Redesign

The most consequential fix in this project: the Business Impact tab's numbers didn't match the uploaded source file, and three SKUs showed the opposite risk conclusion their own numbers supported. Combined with a full visual redesign request.

```
Fix the following - correctness issues first, then visual design:

DATA FIDELITY:
1. Calculate ONLY from the literal parsed contents of demand_and_cost.csv,
   matched by exact SKU + Site. Never generate or default these values -
   show an error instead of substituting invented numbers.
2. Only show a SKU-Site if it has a matching row in demand_and_cost.csv.
3. Verify the sign of Delta Units matches the Risk Flag label for every
   row - this was inverted for several rows in the last version.
4. Fix "Update Demand & Cost CSV" so it actually replaces the dataset and
   recalculates immediately.

VISUAL DESIGN (apply to both tabs):
5. Remove the gradient "hero banner" style section - use a plain header.
6. Cut icon/symbol use to only where it adds real information.
7. Restrict color to semantic meaning only: red = stockout risk, amber =
   excess carrying cost, green = healthy/no gap, neutral otherwise.
8. Dense, table-first layout - thin borders instead of colorful cards.
```

```
9. Apply the same visual language to the IAT Results tab so both tabs
   look like one consistent, professional product.
```

Outcome: All data-fidelity issues resolved and re-verified against the hand-built answer key (Defects 5-6 closed). Visual redesign applied consistently across both tabs. This was the final fix before UAT sign-off.

Summary

Seven build/fix prompts were used to take this project from initial concept to a fully tested v1. Full defect details, expected-vs-actual figures, and sign-off status are in the companion UAT Test Cases & Results document.